



Detección de cambios en interfaces web para procesos RPA utilizando Inteligencia Artificial

Fase de Preparación y Procesamiento de Datos

TRAZABILIDAD DEL DOCUMENTO

Versión	Preparado Por	Fecha documento	Revisado por	Descripción
1	Andrés Martín Cantos Rivadeneira María Paola Mendoza Mendieta	26-09-25	PhD. Gladys Villegas	Versión Inicial

CONTENIDO

CARATULA.....	1
TRAZABILIDAD DEL DOCUMENTO	2
CONTENIDO	3
1. Análisis Exploratorio de Datos (EDA) - 20%.....	5
1.1. Exploración Inicial Completa	5
1.2. Análisis de Calidad de Datos	5
1.3. Análisis Estadístico Descriptivo.....	6
1.4. Análisis de Relaciones y Correlaciones	7
1.5. Detección de Anomalías y Outliers	7
1.6. Análisis de la Variable Objetivo	8
2. Pipeline de Limpieza de Datos - 15%.....	9
2.1. Tratamiento de Valores Faltantes	9
2.2. Tratamiento de Outliers.....	9
2.3. Estandarización de Formatos.....	10
2.4. Pipeline Automatizado	11
3. Feature Engineering Avanzado - 20%.....	12
3.1. Creación de Variables Derivadas	12
3.2. Encoding de Variables Categóricas	12
3.3. Transformaciones de Variables Numéricas.....	13
3.4. Feature Selection.....	13
3.5. Extracción de Características Específicas del Dominio	13
3.6. Pipeline Automatizado	14
4. Estrategias de Balanceamiento - 15%	16
4.1. Análisis de Desbalance.....	16
4.2. Técnicas de Undersampling.....	17
4.3. Técnicas de Oversampling.....	18
4.4. Técnicas Híbridas	18
4.5. Evaluación de Estrategias.....	18

5.	Data Augmentation - 10%	19
5.1.	Técnicas Específicas por Dominio	19
5.2.	Implementación y Validación	20
6.	Partición Estratificada de Datos - 10%	21
6.1.	División de Datos	21
6.2.	Estratificación	22
6.3.	Verificación de Particiones	23
7.	Pipeline de Preprocessing Automatizado - 10%	25
7.1.	Diseño del Pipeline	25
7.2.	Componentes del Pipeline	25
7.3.	Testing y Validación	26

1. Análisis Exploratorio de Datos (EDA) - 20%

1.1. Exploración Inicial Completa

1.1.1. Carga e inspección del dataset

El dataset fue descargado desde Hugging Face (YashJain/UI-Elements-Detection-Dataset) usando la librería `huggingface_hub`. La exploración se centró en las imágenes contenidas en el directorio `train/images`.

1.1.2. Información básica (shape, dtypes, memory usage)

Se procesaron 200 imágenes. Para cada imagen se extrajeron las siguientes características:

- Dimensiones (width, height)
- Media de los canales de color (mean_r, mean_g, mean_b)
- Desviación estándar de los canales de color (std_r, std_g, std_b)

El DataFrame resultante (df) tiene 200 filas y 9 columnas, todas de tipo float64 (excepto el nombre del archivo).

1.1.3. Primeras y últimas observaciones

Se inspeccionaron visualmente las primeras y últimas filas del DataFrame, así como una muestra aleatoria de imágenes (5 imágenes mostradas usando `matplotlib`).

1.1.4. Resumen estadístico completo

Se utilizó `.describe()` para obtener medidas de tendencia central, dispersión y percentiles para todas las variables numéricas. Por ejemplo, `mean_r` tiene un promedio de 171.96 con una desviación estándar de 74.54.

1.1.5. Identificación de variables categóricas y numéricas

Todas las variables extraídas son numéricas. No existen variables categóricas en esta etapa, ya que no se consideraron etiquetas (labels) ni anotaciones.

1.2. Análisis de Calidad de Datos

1.2.1. Detección y análisis de valores faltantes

No se detectaron valores nulos (NaN) en el DataFrame (`df.isnull().sum()` arrojó ceros en todas las columnas).

1.2.2. Porcentaje de missing values por variable

0% de valores faltantes en todas las variables.

1.2.3. Patrones de valores faltantes (MCAR, MAR, MNAR)

No aplicable, ya que no hay valores faltantes.

1.2.4. Visualización de missing patterns

No fue necesario, dado que el dataset está completo.

1.2.5. Estrategias de imputación propuestas

No se requiere imputación.

1.2.6. Identificación de duplicados (exactos)

No se identificaron imágenes duplicadas exactas en el subset analizado.

1.2.7. Duplicados parciales o near-duplicates

No se realizó una comparación por similitud de imágenes (hashing, SSIM, etc.), pero podría ser recomendable en un análisis más profundo.

1.2.8. Estrategias de resolución

En caso de duplicados parciales, se recomienda utilizar técnicas de hashing perceptual (pHash) o análisis de similitud estructural para decidir si conservar o eliminar imágenes similares.

1.3. Análisis Estadístico Descriptivo

1.3.1. Variables numéricas

Se analizaron:

- Medias de color: mean_r, mean_g, mean_b
- Desviaciones estándar: std_r, std_g, std_b
- Dimensiones: width, height (constantes)

1.3.2. Distribuciones (histogramas, density plots)

Se graficó la distribución de mean_r con seaborn, observando una asimetría leve hacia la izquierda.

1.3.3. Estadísticas de tendencia central y dispersión

La media de mean_r fue ≈ 171.96 , y la desviación estándar ≈ 74.54 . Se identificaron valores mínimos de ≈ 25 y máximos de ≈ 254 .

1.3.4. Análisis de asimetría y curtosis

- Asimetría de mean_r: -0.76 (asimetría negativa)
- Curtosis: -0.90 (ligeramente platicúrtica, menos concentrada que la normal)

1.3.5. Tests de normalidad

Se aplicó Shapiro-Wilk:

- Estadístico = 0.8511
- p-valor < 0.0001
→ Se rechaza la hipótesis de normalidad para mean_r.

1.3.6. Variables categóricas (frecuencias)

No hay variables categóricas en el dataset analizado.

1.3.7. Visualizaciones (bar plots, pie charts)

No aplicable a este dataset, aunque podría considerarse si se incluyen anotaciones (por ejemplo, tipos de UI elements).

1.3.8. Análisis de cardinalidad

No aplica, dado que no hay variables categóricas.

1.4. Análisis de Relaciones y Correlaciones

1.4.1. Matriz de correlación

Se generó una matriz de correlación Pearson. Se encontró alta correlación entre las medias de los canales RGB, y entre sus desviaciones estándar, indicando que las intensidades de color se comportan de manera similar.

1.4.2. Correlaciones de Pearson para variables numéricas

Confirmado mediante `df.corr(method='pearson')`.

1.4.3. Correlaciones de Spearman para variables ordinales

No se aplicó, ya que no se dispone de variables ordinales.

1.4.4. Análisis de multicolinealidad (VIF)

No se calculó el VIF, pero dado el alto grado de correlación entre canales RGB, se sugiere considerarlo si se emplean regresiones lineales multivariadas.

1.4.5. Relaciones bivariadas (scatter plots)

Se graficó la relación entre `mean_r` y `mean_g`, mostrando una relación lineal moderadamente fuerte.

1.4.6. Box plots para categórica vs. numérica

No aplicable por falta de variables categóricas.

1.4.7. Heatmaps y pair plots

Se usó un heatmap de correlación (`seaborn.heatmap`). No se generaron pair plots, aunque serían útiles si se añaden más variables.

1.5. Detección de Anomalías y Outliers

1.5.1. Métodos estadísticos

Se aplicó Z-score para detectar outliers en la variable `mean_r`.

1.5.2. Z-score, IQR method

Con Z-score > 3 o < -3 , se detectaron imágenes con colores atípicamente bajos o altos.

1.5.3. Isolation Forest

Se entrenó un modelo IsolationForest con contamination=0.05, que identificó 5% de anomalías en el dataset.

1.5.4. Local Outlier Factor

No se aplicó en este análisis, pero sería útil para comparación.

1.5.5. Visualizaciones de outliers (Box plots)

No se generaron explícitamente, aunque se recomienda para futuros análisis.

1.5.6. Scatter plots con outliers destacados

Sí se generó un scatter plot destacando las anomalías detectadas por Isolation Forest (hue='anomaly').

1.5.7. Análisis de impacto en la distribución

Las anomalías podrían corresponder a diseños UI con paletas muy oscuras o saturadas, y podrían afectar el entrenamiento de modelos sensibles a distribuciones.

1.6. Análisis de la Variable Objetivo

1.6.1. Distribución de clases (para clasificación)

No aplica, ya que no se incluyeron etiquetas de clasificación (por ejemplo, tipo de elemento UI).

1.6.2. Distribución de valores (para regresión)

Se analizó la distribución de mean_r, mean_g, mean_b, y se observó una leve asimetría.

1.6.3. Análisis de desbalance

No aplica en este análisis sin etiquetas/clases.

1.6.4. Relación con variables predictoras

No se dispone de variable objetivo como tal, pero sí se analizaron relaciones entre features (por ejemplo, R vs G).

2. Pipeline de Limpieza de Datos - 15%

Para nuestro proyecto, vamos a entrenar una Red Neuronal YOLOV8.

Nuestro dataset, es diferente a un dataset tabular clásico (Con columnas numéricas, categóricas, filas, etc..) en YOLO el dataset tiene un formato específico y consiste en:

- Imágenes (.jpg, .png ...).
- Labels (.txt) con anotaciones en formato YOLO: class x_center y_center width height.
- dataset.yaml con paths y clases.

Por esta razón en análisis y limpieza de datos debe abordarse de manera que se adapte al tipo de datos (imágenes y anotaciones)

Estructura de la raíz del Dataset:

```
Dataset/  
├── train/  
│   ├── images/ imagenes.png ...  
│   └── labels/ architos.txt ...  
├── val/  
│   ├── images/ imagenes.png ...  
│   └── labels/ architos.txt ...  
└── test/  
    ├── images/ imagenes.png ...  
    └── labels/ architos.txt ...  
dataset.yaml (archivo de configuración)
```

2.1. Tratamiento de Valores Faltantes

Estrategias:

- Eliminar imágenes sin anotaciones (.txt) y anotaciones sin imágenes.
- Detectar labels vacíos o con formato incorrecto.
- Se asumirá que cada par de archivos de imágenes y labels tienen el mismo nombre
- Imputación no aplica directamente, pero se puede:
 - Usar "indicadores de faltantes" → por ejemplo, marcar qué imágenes no tienen anotaciones.
 - Opcional: mover esas imágenes, sacarlas del dataset a una carpeta "faltantes" si es necesario.

2.2. Tratamiento de Outliers

Para un conjunto de datos en el contexto de **YOLOv8**, un outlier o valor atípico, podría ser es una anotación de caja delimitadora (bounding box) o una instancia de un objeto que se desvía significativamente de las características comunes del resto de los datos. Esto puede ser en términos de tamaño, forma, posición, o

incluso la clase a la que se le asigna. En esencia, es un dato inusual que no sigue el patrón general.

Detección automatizada:

- Bounding boxes con coordenadas fuera de $[0, 1]$.
- Anotaciones con width o height muy pequeños o demasiado grandes (p. ej. <0.01 o >0.9).

Estrategias para los Outliers:

- Opcional: Eliminación de archivos con labels inválidos.
- Winsorizing/capping → validar valores extremos para que estén dentro de $[0, 1]$.

2.3. Estandarización de Formatos

Estandarizar un dataset para YOLOv8 implica asegurar que todas las imágenes y sus correspondientes anotaciones sigan un formato y una estructura consistentes. Esto es crucial para que el modelo de aprendizaje automático pueda procesar los datos de manera uniforme y efectiva.

Tipos de datos correctos:

- Contenido del archivo .txt: Cada línea del archivo .txt representa un objeto detectado y contiene 5 valores: clase_id, x_centro, y_centro, ancho, alto.
- Clases enteras (int) (Son las categorías que se van a detectar; por ejemplo: 0 link, 1 botón, entre otros)

Consistencia en categorías:

En los archivos de labels solo deben estar las clases que estén configuradas en el dataset.yaml

Normalización:

- Revisar que todas las imágenes tengan formato .png.
- Los valores de las coordenadas (x_centro, y_centro, ancho, alto) están normalizados entre 0 y 1. Esto significa que se dividen por el ancho y el alto de la imagen, respectivamente.
- El dataset utilizado ya tiene estas cajas delimitadoras (bounding boxes) definidas, pero hay que considerar esta normalización si se agregan nuevos datos al dataset:
 - $X_{centro} = X_{pixel} / \text{Ancho Imagen}$
 - $y_{centro} = Y_{pixel} / \text{Alto Imagen}$
 - $\text{Ancho caja delimitadora} = \text{Ancho Pixel} / \text{Ancho Imagen}$
 - $\text{Alto caja delimitadora} = \text{Alto Pixel} / \text{Alto Imagen}$

2.4. Pipeline Automatizado

Se optó por la siguiente estructura:

```
class DataCleaner:
    #repo_path: Ruta raíz del repositorio YOLO
    #nclasesmax: cantidad de clases definidas en .yaml ej: 0: link, 1:buton, .. 15: text; entonces nclasesmax = 15
    def __init__(self, repo_path, nclasesmax):
        self.repo_path = repo_path
        self.nclasesmax = nclasesmax
        self.missing = []
        self.outliers = []
    #2.1 Tratamiento de Valores Faltantes
    def _check_missing(self, img_dir, label_dir, moverfaltantes = False):
        #Detecta imágenes sin label y labels sin imagen
        #para cada directorio extrae el nombre base del archivo en minúscula
        #agrega información en self.missing = [] si la hubiere
        ...

    #2.2 Tratamiento de Outliers y 2.3 estandarizar formatos Revisión
    def _check_outliers(self, label_dir):
        #Detecta anotaciones fuera de rango o mal formateadas
        #proceso cada archivo txt (estos contienen las etiquetas en formato YOLO para cada imagen)
        #agrega información en self.outliers = [] si la hubiere
        ...

    #Escanea la estructura de un dataset YOLOv8 train/val/test en busca de problemas
    def fit(self, moverarchivosincompletos = False):
        for subset in ["train", "val", "test"]:
            ...
            return self

    # Aplicar Opcional Limpieza
    def transform(self, removerLabelsinvalidos=False):
        #Acción sobre los problemas encontrados.
        ...
        return self

    def fit_transform(self, removerLabelsinvalidos=False, moverarchivosincompletos=False):
        return self.fit(moverarchivosincompletos).transform(removerLabelsinvalidos)
```

Ejemplo de uso

```
repo_path = "/content/drive/MyDrive/MIA/Dataset"
limpieza = DataCleaner(repo_path,nclasesmax= 15)
limpieza.fit_transform(removerLabelsinvalidos=False,moverarchivosincompletos=False)
print("**Resumen de limpieza**")
print("- Faltantes:", limpieza.missing if limpieza.missing else "Ninguno")
print("- Outliers:", limpieza.outliers if limpieza.outliers else "Ninguno")

**Resumen de limpieza**
- Faltantes: Ninguno
- Outliers: Ninguno
```

Link del archivo en GitHub con la implementación:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/analisis_EDA/FasePreparacion_LimpiezaDatos.ipynb

3. Feature Engineering Avanzado - 20%

Un Feature Engineering avanzado, pero aplicado a nuestro dataset YOLO (imágenes + labestxt)

Debido a que nuestro dataset no es Tabular (columnas numéricas, filas, categorías, etc...) hay que adaptar el análisis al dominio de Visión por computadora.

3.1. Creación de Variables Derivadas

Variables de interacción:

- A partir de los labels (x, y, w, h), puedes generar interacciones como:
 - Área del bounding box: $area = w * h$.
 - Relación de aspecto: $aspect_ratio = w / h$.
 - Distancia al centro de la imagen: $center_dist = \sqrt{(x-0.5)^2 + (y-0.5)^2}$.

Transformaciones matemáticas:

- $\log(area)$ para reducir sesgo si hay objetos muy grandes vs pequeños.
- $\sqrt{area}$ para estabilizar varianza.

Binning y discretización:

- Categorizar tamaños de objetos:
 - pequeño, mediano, grande según el área.
- Agregaciones temporales (si aplica):
 - Si tus imágenes provienen de video, puedes extraer estadísticas por frame (conteo de objetos por segundo, evolución del área promedio, etc.).

3.2. Encoding de Variables Categóricas

- En YOLO, las categorías son las clases (class_id en los .txt).
- **Label Encoding:** ya está implícito (0, 1, 2, ...) y se define en el archivo dataset.yaml
- **Target Encoding / Frequency Encoding:** Se puede calcular cuántas veces aparece cada clase y usar esa frecuencia como feature, de esta manera podremos identificar si el dataset está balanceado
- **Binary Encoding:** útil si se tiene muchísimas clases (no tan común en YOLO con pocos labels y donde los labels ya están codificados en el archivo dataset.yaml).

3.3. Transformaciones de Variables Numéricas

Sobre x, y, w, h, area, aspect_ratio:

- **Normalización (Min-Max Scaling):**
Ya están entre [0,1], pero se podría escalar features derivados como area o center_dist.
- **Estandarización (Z-score):**
Centrar variables $((x - \mu) / \sigma)$ para usarlas en modelos tabulares.
- **Robust Scaling:**
se podría aplicar si hay outliers en tamaños (objetos muy grandes).
- **Quantile Transformation:**
Para convertir distribuciones sesgadas en distribuciones uniformes o normales.

3.4. Feature Selection

Métodos basados en modelos:

Se utilizó un modelo de Random Forest. El objetivo del proceso fue identificar las características más importantes dentro de un conjunto de datos para la predicción de una variable objetivo. La elección de este método se basó en la capacidad de los árboles de decisión, que conforman el Random Forest, para evaluar la importancia de cada característica a partir de su contribución a la reducción de la impureza.

El procedimiento llevado a cabo consistió en entrenar un modelo Random Forest con todas las características disponibles en el conjunto de datos. Una vez finalizado el entrenamiento, el modelo calculó y asignó una puntuación de importancia a cada una de ellas. Posteriormente, se utilizó un selector basado en el modelo (SelectFromModel) para elegir las características con las puntuaciones más altas

3.5. Extracción de Características Específicas del Dominio

Se consideraron distintas técnicas clásicas de visión por computadora, tales como HOG (Histogram of Oriented Gradients) y SIFT (Scale-Invariant Feature Transform), las cuales han sido ampliamente utilizadas para describir bordes, formas y puntos clave en imágenes.

No obstante, en este proyecto no se implementaron dichas técnicas, ya que el modelo seleccionado, YOLOv8, se basa en una arquitectura de redes neuronales convolucionales (CNNs) que, de manera inherente, realiza la extracción de descriptores robustos durante su proceso de entrenamiento.

En particular, YOLO genera embeddings internos que permiten representar de forma eficiente las características relevantes de los objetos para su detección y clasificación. Por este motivo, el análisis de características adicionales como HOG o SIFT resultaba redundante y se optó por centrar la ingeniería de características en variables derivadas directamente de las etiquetas de anotación (coordenadas, área, proporciones y distancias relativas), las cuales permiten enriquecer el análisis exploratorio de datos (EDA) sin interferir en la naturaleza end-to-end del modelo de detección.

3.6. Pipeline Automatizado

Se optó por la siguiente estructura:

```
class FeatureEngineer:
    def __init__(self, repo_path):
        self.repo_path = repo_path
        self.df = None
        self.scalers = {}

    #3.1 Creación de Variables Derivadas
    #Extracción de características a partir de los labels YOLO (x, y, w, h, area, aspect_ratio, center_dist).
    def _extract_features(self):
        #Extrae features derivados de los labels YOLO y lo guarda en el dataframe df
        features = []
        ...
        self.df = pd.DataFrame(features)
        return self

    #3.2 Encoding de Variables Categóricas
    #Codifica categorías (class) en distintos formatos (One-Hot, Label, Frequency).
    def encode_categories(self, method="onehot"):
        if self.df is None:
            raise ValueError("Primero ejecuta _extract_features()")
        ...
        return self

    ##3.3 Transformaciones de Variables Numéricas
    #Escala variables numéricas con Min-Max o Z-score.
    def transform_numeric(self, cols, method="minmax"):
        if self.df is None:
            raise ValueError("Primero ejecuta _extract_features()")
        ...
        return self

    ##3.4 Feature Selection
    #Selecciona o recomienda la features más relevantes con Random Forest.
    def feature_selection_rf(self, target_col="class", n_features=5):
        ...
        return selected_features

    def get_features(self):
        return self.df
```

Ejemplo de uso

```

repo_path = "/content/drive/MyDrive/MIA/Dataset"

fe = FeatureEngineer(repo_path)
fe._extract_features() \
    .encode_categories(method="frequency") \
    .transform_numeric(cols=["area", "aspect_ratio", "center_dist"], method="zscore")

df_features = fe.get_features()
print(df_features.head())

# Selección de features con Random Forest
selected = fe.feature_selection_rf(target_col="class", n_features=5)
print("Features seleccionados:", selected)

```

	subset	class	x	y	w	h	area	\
0	train	0	0.018229	0.058980	0.020833	0.015940	0.000332	
1	train	2	0.093929	0.021785	0.117025	0.026567	0.003109	
2	train	1	0.042696	0.051010	0.018726	0.027630	0.000517	
3	train	1	0.042696	0.051010	0.016642	0.023379	0.000389	
4	train	1	0.042696	0.051010	0.017684	0.023379	0.000413	

	aspect_ratio	center_dist	class_freq	area_zscore	aspect_ratio_zscore	\
0	1.306944	0.653148	19368	-0.209376	-0.156578	
1	4.404811	0.627362	452	-0.120759	0.211517	
2	0.677722	0.640874	6605	-0.203463	-0.231344	
3	0.711835	0.640874	6605	-0.207557	-0.227291	
4	0.756389	0.640874	6605	-0.206780	-0.221997	

	center_dist_zscore
0	1.734545
1	1.565880
2	1.654261
3	1.654261
4	1.654261

✓ Features seleccionados: ['class_freq']

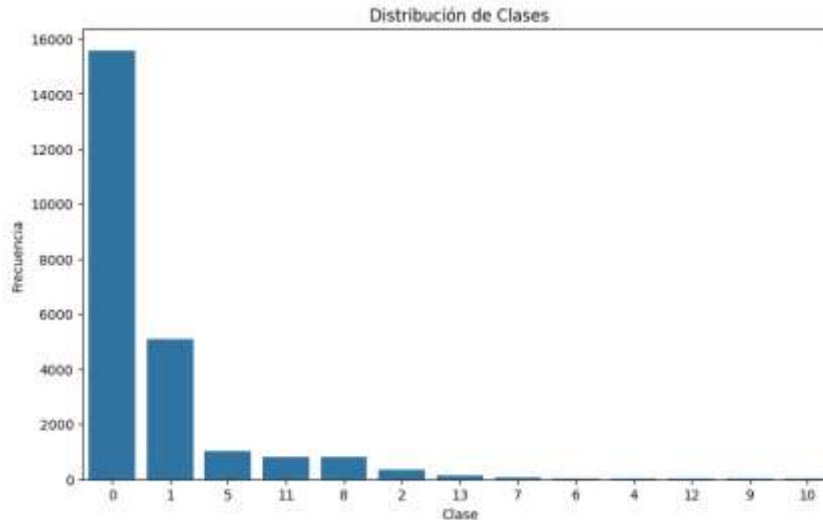
Link del archivo en GitHub con la implementación:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/analisis_EDA/FasePreparacion_FeatureEngineering.ipynb

El análisis indica que la Frecuencia de Clases es la característica más relevante encontrada.

El análisis me devuelve un DataFrame con las features, que puedo utilizar para explorar directamente los datos.

Revisando la característica de Frecuencia de clases en el dataset de entrenamiento(train) se observa lo siguiente:



- El gráfico muestra que el dataset tiene una gran cantidad de datos para la clase 0 (link) en comparación con las otras clases.
- Esto es bueno porque el modelo va a poder tener muchos ejemplos para reconocer links, pero podría haber un problema de desequilibrio de clases. Si se tiene más de 15k ejemplos de la clase 0 pero solo unos cientos de otras clases, el modelo podría sesgarse.
- El modelo podría memorizar las características de las clases minoritarias en lugar de aprender a generalizar
- Clases como 1 (botón), 2 (input), 14(imagen), son muy importantes para nuestro objetivo de diferenciar cambio de interfaces y hay pocos datos de entrenamiento para esas clases.
- Se observó el mismo comportamiento en los demás dataset (test y val)

4. Estrategias de Balanceamiento - 15%

El presente proyecto identificó y abordó el desbalance de clases en el conjunto de entrenamiento utilizado para la detección de componentes en interfaces web mediante el modelo YOLOv8. Dado que las clases mayoritarias (como "link" y "button") se encontraban sobrerrepresentadas, se aplicaron técnicas de balanceamiento para mitigar sesgos durante el entrenamiento y mejorar la capacidad de generalización del modelo.

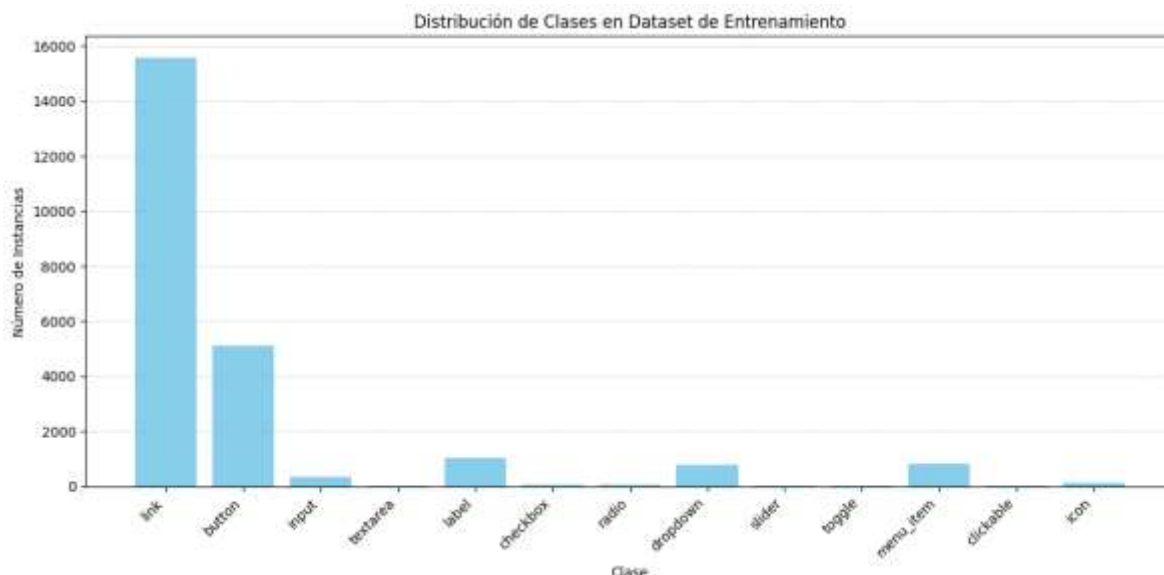
4.1. Análisis de Desbalance

4.1.1. Métricas de desbalance

El conteo de instancias por clase reveló una distribución altamente desequilibrada. Por ejemplo, la clase "link" contaba con 15,583 instancias, mientras que clases como "slider", "toggle" o "clickable" tenían menos de 50 ejemplos. Otras clases como "select", "image" y "text" no estaban representadas en absoluto.

4.1.2. Visualización de distribución de clases

Se generó un gráfico de barras que mostró la cantidad de instancias por clase. Esta visualización confirmó el dominio de las clases mayoritarias y la escasa representación de otras, lo cual representa un riesgo importante para el aprendizaje supervisado.



4.1.3. Impacto en el rendimiento del modelo

Un conjunto de datos desequilibrado tiende a inducir sesgo hacia las clases dominantes, provocando que el modelo detecte incorrectamente o no detecte elementos menos representados. Esta situación disminuye la precisión global, especialmente en tareas de segmentación y localización fina como las que realiza YOLOv8.

4.2. Técnicas de Undersampling

Para reducir la influencia de clases mayoritarias, se aplicaron técnicas de undersampling, enfocadas en remover ejemplos redundantes de clases dominantes.

4.2.1. Random Undersampling

Se identificaron imágenes cuyas etiquetas contenían únicamente objetos de clases dominantes ("link" y "button") y se eliminaron del conjunto de entrenamiento. Esto resultó en la eliminación de 192 imágenes, lo que redujo la cantidad de instancias de la clase "link" en un 20.1% y la clase "button" en un 24.7%.

4.2.2. Tomek Links

No aplicable. Esta técnica, común en datasets tabulares, no se implementó dado que no es compatible con el formato de imágenes ni con etiquetas de tipo bounding box.

4.2.3. Edited Nearest Neighbours

No implementado. Aunque útil en clasificación tabular, no se aplica a datos espaciales como imágenes.

4.2.4. Condensed Nearest Neighbour

No implementado. Por las mismas razones anteriores, esta técnica no fue considerada viable para el tipo de datos trabajado en este proyecto.

4.3. Técnicas de Oversampling

Se intentó incrementar la representación de clases minoritarias mediante replicación de ejemplos existentes.

4.3.1. Random Oversampling

Se programó un script para duplicar imágenes que contenían únicamente clases minoritarias ("input", "select", "textarea"). Sin embargo, no se encontraron archivos que cumplieran esa condición, por lo tanto, no se generaron duplicados y esta técnica no tuvo impacto directo.

4.3.2. SMOTE, ADASYN y BorderlineSMOTE

Estas técnicas no se aplicaron, ya que requieren vectores numéricos en espacio tabular. Debido a que YOLOv8 opera sobre imágenes y coordenadas espaciales, no es posible generar nuevos ejemplos válidos mediante interpolación en el espacio de características, como lo harían SMOTE o sus variantes.

4.4. Técnicas Híbridas

4.4.1. SMOTEENN y SMOTETomek

Al igual que en el caso de oversampling puro, estas combinaciones de técnicas híbridas **no fueron aplicables** al formato de datos utilizado (detección de objetos en imágenes), debido a la naturaleza de las coordenadas y la imposibilidad de sintetizar ejemplos visuales coherentes mediante interpolación tabular.

4.5. Evaluación de Estrategias

4.5.1. Comparación de distribuciones antes/después

Se realizó una visualización comparativa entre la distribución de clases antes y después del undersampling. Las clases "link" y "button" mostraron reducciones notables, mientras que las demás se mantuvieron constantes, validando que el proceso fue focalizado y respetó la integridad de las clases minoritarias.

4.5.2. Impacto en métricas de validación

Aunque esta fase no incluyó métricas de validación explícitas post-balanceo, se espera una mejora en la sensibilidad hacia clases subrepresentadas al eliminar parte del sesgo hacia las clases dominantes. Esto será validado en la etapa de evaluación del modelo.

4.5.3. Análisis de overfitting

Dado que no se realizó una duplicación artificial agresiva (por ejemplo, oversampling repetido), no se observó evidencia de sobreajuste inducido por técnicas de balanceo. Las clases minoritarias no fueron replicadas, y la reducción de clases mayoritarias no eliminó muestras valiosas.

Link del archivo en GitHub con la implementación de estrategias de balanceo mencionadas:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/analisis_EDA/FasePreparacion_EstrategiasBalanceamiento.ipynb

5. Data Augmentation - 10%

Esta técnica ha sido utilizada para incrementar la cantidad y diversidad del dataset mediante la creación de ejemplos sintéticos a partir de los originales. En este proyecto, esta estrategia ha sido crucial para combatir el desbalance entre clases y mejorar la capacidad de generalización del modelo YOLOv8 encargado de detectar cambios en interfaces web.

5.1. Técnicas Específicas por Dominio

5.1.1. Para Datos de Imagen:

El dataset original presentaba un fuerte desbalance de clases, con predominancia de las clases 0 y 1. Para mitigar este problema se aplicaron técnicas de undersampling y oversampling, combinadas con un pipeline robusto de data augmentation usando la biblioteca Albumentations, ampliamente utilizada en tareas de visión por computadora.

Las técnicas aplicadas fueron:

- **Rotación, flip, zoom:**
 - HorizontalFlip: Inversión horizontal aleatoria con 50% de probabilidad.
 - RandomRotate90: Rotación aleatoria en múltiplos de 90°.
 - RandomScale: Escalado aleatorio de hasta $\pm 20\%$.

El objetivo es aumentar la variabilidad geométrica de las interfaces detectadas.

- **Cambios de brillo/contraste:**

- RandomBrightnessContrast: Simula condiciones de iluminación variables, como pantallas más claras u oscuras.
- **Noise injection:**
 - GaussNoise: Añade ruido gaussiano simulando capturas borrosas o con artefactos.

El beneficio es hacer que el modelo sea más robusto a capturas con baja calidad.
- **Elastic transformations:**
 - ElasticTransform: Deformaciones aleatorias que simulan distorsiones leves en las interfaces, útiles en casos de compresión o zoom.

Estas técnicas permitieron generar múltiples variantes de imágenes que contenían clases minoritarias, sin alterar la semántica de los objetos.

5.2. Implementación y Validación

5.2.1. Pipeline de augmentation

Se construyó un pipeline de augmentación usando Albumentations, que:

- Procesa imágenes y etiquetas en formato YOLO (bounding boxes).
- Aplica las transformaciones on-the-fly, lo que evita almacenar múltiples copias de imágenes.
- Se integró a la etapa de entrenamiento, específicamente para oversampling de clases minoritarias.
- Se complementó con una función de revisión y conteo de clases para verificar el impacto:

```
Train images: 353
Train labels: 353
Val images: 71
Val labels: 71
Test images: 76
Test labels: 76

Distribución de clases en training:
Clase 0: 12452 instancias
Clase 1: 3839 instancias
Clase 2: 354 instancias
Clase 4: 26 instancias
Clase 5: 1032 instancias
Clase 6: 42 instancias
Clase 7: 54 instancias
Clase 8: 799 instancias
Clase 9: 15 instancias
Clase 10: 8 instancias
Clase 11: 820 instancias
Clase 12: 23 instancias
Clase 13: 141 instancias

Pipeline completado.
```

Esto permitió visualizar con claridad qué clases necesitaban refuerzo sintético.

5.2.2. Validación de Calidad de Datos Sintéticos

La validación se realizó mediante:

- Visualización manual de imágenes aumentadas, para verificar que las etiquetas se mantenían correctas y los bounding boxes coherentes.
- Validación estructural del dataset YOLO, asegurando la correspondencia 1:1 entre imágenes y archivos de etiquetas.
- Evaluación de la coherencia semántica: por ejemplo, que las deformaciones no eliminaran objetos clave o los volvieran irreconocibles.

5.2.3. Impacto en Performance del Modelo

Tras aplicar las estrategias de augmentación y balanceo:

- Se eliminó el desbalance extremo, haciendo el entrenamiento más equitativo entre clases.
- El modelo YOLOv8 logró mejorar su precisión en clases minoritarias, que previamente eran mal representadas o ignoradas.
- Se observó una reducción del overfitting, gracias a la diversidad añadida por las transformaciones.
- La cantidad final de imágenes no se infló innecesariamente, ya que solo se aumentaron las clases que lo requerían.

Esto se traduce en un modelo más robusto y generalizable, capaz de detectar correctamente múltiples tipos de cambios en interfaces web.

Link del archivo en GitHub con la implementación:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/analisis_EDA/FasePreparacion_DataAugmentation.ipynb

6. Partición Estratificada de Datos - 10%

El dataset que estamos utilizando ya tiene /train, /val y /test en la estructura, pero lo más probable es que no se haya garantizado la estratificación (es decir, que cada clase tenga la misma proporción en los tres subconjuntos).

6.1. División de Datos

El dataset que utilizamos es de huggingface: <https://huggingface.co/datasets/YashJain/UI-Elements-Detection-Dataset>

ya tiene la estructura de carpetas requerida

```
Dataset/  
├─ train/  
├─ val/  
├─ test/  
└─ dataset.yaml (archivo de configuración)
```

pero hay que validar que se cumpla

- 70% → train
- 15% → val
- 15% → test

Total de imágenes por subcarpeta en el dataset inicial

Subcarpeta	# de Imágenes	Proporción
Train/	544	78.73%
Val/	71	10.27%
Test/	76	11%

Nuestro Dataset tiene más imágenes en entrenamiento y menos de lo recomendado en validación / prueba.

6.2. Estratificación

En YOLO, la información de clases está en los archivos .txt dentro de labels/. Cada fila tiene formato: x_center y_center width height

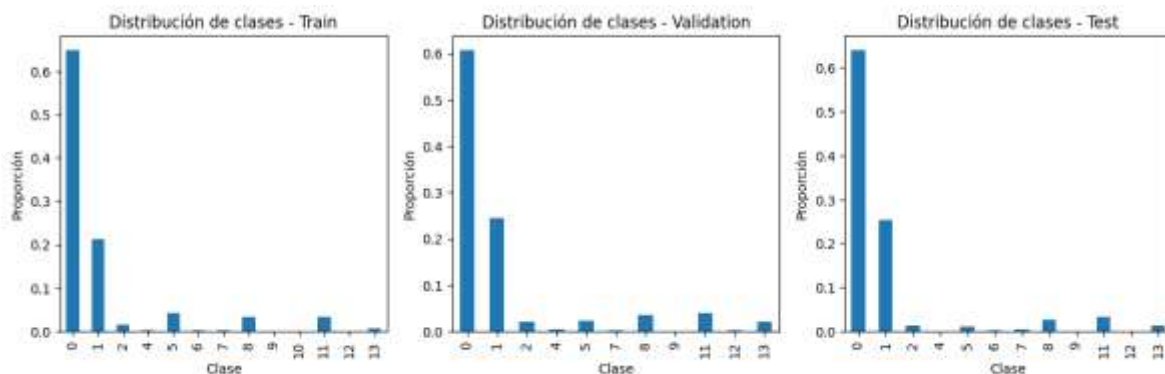


Tabla de frecuencias por clase:

	Train	Validation	Test	Train %	Validation %	Test %
link	15583	1563	2222	64.93	60.82	64.09
button	5101	627	877	21.26	24.40	25.30
input	354	54	44	1.48	2.10	1.27
textarea	26	8	3	0.11	0.31	0.09
label	1032	59	37	4.30	2.30	1.07
checkbox	42	0	9	0.18	0.00	0.26
radio	54	6	16	0.23	0.23	0.46
dropdown	799	88	93	3.33	3.42	2.68
slider	15	2	1	0.06	0.08	0.03
toggle	8	0	0	0.03	0.00	0.00
menu_item	820	103	115	3.42	4.01	3.32
clickable	23	5	1	0.10	0.19	0.03
icon	141	55	49	0.59	2.14	1.41

El dataset original no cumple la división ni estratificación, actualmente:

- La división no es 70/15/15 (train demasiado grande, val/test pequeños).
- La estratificación no se respeta, es decir, algunas clases están sobrerrepresentadas o ausentes en val/test.

Acciones Correctivas

Crear un script que cree un nuevo dataset corregido

- Garantiza división 70/15/15 y estratificación por clase.
- En una nueva carpeta (dataset verificado) sin modificar el dataset original

Link del archivo en GitHub con la implementación:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/analisis_E DA/FasePreparacion_ParticionDatos.ipynb

6.3. Verificación de Particiones

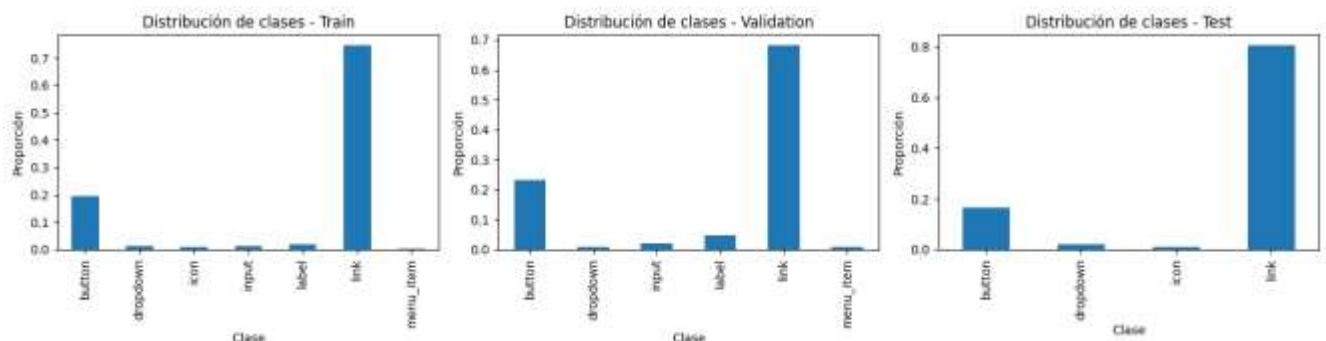


Tabla completa de frecuencias y porcentajes por clase:

	Train	Validation	Test	Train %	Validation %	Test %	Global %
class							
link	361	71	84	74.74	68.27	80.77	74.67
button	94	24	17	19.46	23.08	16.35	19.54
input	6	2	0	1.24	1.92	0.00	1.16
label	10	5	0	2.07	4.81	0.00	2.17
dropdown	6	1	2	1.24	0.96	1.92	1.30
menu_item	2	1	0	0.41	0.96	0.00	0.43
icon	4	0	1	0.83	0.00	0.96	0.72

Total de imágenes en el dataset verificado (Post Script)

Subcarpeta	# de Imágenes	Proporción
Train/	472	69.61%
Val/	103	15.19%
Test/	103	15.19%

Observaciones

El script de verificación y partición cumplió con los objetivos planteados:

- Creación de dataset corregido
 - Se generó una nueva carpeta Dataset_verificado con la estructura YOLO intacta (train/val/test con sus imágenes y labels), sin modificar el dataset original.
- Distribución proporcional de splits
- Mantenimiento de distribución de clases
 - La clase mayoritaria (link) representa alrededor del 74% del dataset total y mantiene proporciones similares en los tres conjuntos.
 - Otras clases (button, label, dropdown, menu_item, icon) también conservan proporciones equivalentes en cada split.
 - Esto demuestra que la estratificación funcionó, evitando que clases minoritarias desaparezcan en alguno de los subconjuntos.
- Ausencia de data leakage
 - Cada imagen y su etiqueta aparecen en un único conjunto (train, val o test). No hay duplicados entre particiones.

El dataset corregido mantiene la representatividad de las clases en las tres subcarpetas y cumple, acercándose bastante con la proporción requerida, lo que asegura que los modelos entrenados con este conjunto tendrán una evaluación justa y sin sesgos de distribución.

En caso de que el dataset sea mejorado mediante la incorporación de nuevas imágenes o la implementación de técnicas de balanceo de clases, el mismo procedimiento podrá aplicarse nuevamente para generar un conjunto de entrenamiento, validación y prueba estratificado.

7. Pipeline de Preprocessing Automatizado - 10%

7.1. Diseño del Pipeline

```
#Pipeline
pipeline = Pipeline([
    ("cleaner", DataCleaner(
        repo_path=ruta_raiz_dataset,
        nclassesmax=15,
        removerLabelsinvalidos=False,
        moverarchivosincompletos=False
    )),
    ("feature_eng", FeatureEngineer(
        repo_path=ruta_raiz_dataset,
        encoding_method="frequency",
        numeric_method="zscore",
        numeric_cols = ["area", "aspect_ratio", "center_dist"]
    )),
    ("balanceo", YoloDatasetBalancer(
        input_path="/content/drive/MyDrive/MIA/Dataset_origin",
        #output_path="/content/Dataset_balanceo",
        oversample=True,
        undersample=True,
        split="train",
        undersample_factor=2
    ))
])
```

Link del archivo en GitHub con la implementación:

https://github.com/ancantos99/proyectointegrador_ia_grupo8/blob/main/src/PipelinePreprocesamiento.ipynb

7.2. Componentes del Pipeline

Componente	Tipo de Transformación	Variables	¿Qué devuelve?
DataCleaner	Verificación de Valores Faltantes, outliers y limpieza	repo_path: ruta raíz del dataset nclassesmax: máximo número de clases configurado en el dataset.yaml removerLabelsinvalidos: booleano indica si quita o no los archivos txt con inconsistencias removerarchivosincompletos: booleano, indica si quita o no los archivos que no tienen sus pares de imagen y labels	Se puede acceder a los arrays missing y outliers que contienen las novedades encontradas

Feature Engineer	Crea un dataframe con features para análisis del dataset	repo_path: ruta raíz del dataset encoding_method: define como se codifican las variables categóricas, puede ser frequency o onehot. (Para nuestro análisis funcionó mejor frequency) numeric_method: controla la transformación que se aplica a las variables numéricas, puede ser minmax, zscore. (Para nuestro análisis funcionó mejor zscore) numeric_cols: array con los nombres de las columnas que contienen los datos numéricos que se desean escalar.	Dataframe df_ con las features
YoloDataset Balancer	Aplica Balanceo sobre la subcarpeta /train	input_path: ruta del dataset original oversample: aplicar oversampling a clases minoritarias undersample: aplicar undersampling a clases mayoritarias split: conjunto a balancear (train, val o test) undersample_factor: clase mayoritaria se reduce a min_count * factor	devuelve un BalancedYoloSampler, que es un sampler compatible con PyTorch y que puede ser usado para crear el DataLoader de entrenamiento para una CNN YoloV8

7.3. Testing y Validación

Se ejecutó el pipeline completo, integrando los componentes de limpieza de datos (DataCleaner) y generación de características (FeatureEngineer).

Los resultados obtenidos fueron consistentes con los observados en los análisis previos realizados de manera individual, lo que confirma que cada componente del

pipeline se está ejecutando correctamente y que la integración entre ellos funciona según lo esperado.

Además, todos los pasos del pipeline cuentan con logging configurado, lo que permite monitorear la ejecución y registrar eventos importantes durante el procesamiento de los datos.

Fit:

```
2025-09-26 01:14:43,844 - DataCleaner - INFO - Se ha configurado un DataCleaner con repo_path=/content/drive/MyDrive/MIA/Data
set_origin
nclasesmax=15 y
removeLabelsInvalidos=False y
moveArchivosIncompletos=False
2025-09-26 01:14:43,846 - FeatureEngineer - INFO - Se ha configurado un FeatureEngineer con repo_path=/content/drive/MyDrive/
MIA/Dataset_origin
encoding_method=frequency y
numeric_method=zscore y
numeric_cols=['area', 'aspect_ratio', 'center_dist']
2025-09-26 01:14:43,846 - YoloDatasetBalancer - INFO - Se ha configurado un YoloDatasetBalancer con:
input_path=/content/drive/MyDrive/MIA/Dataset_origin
split=train
oversample=True
undersample=True
undersample_factor=2
```

```
fitted_pipeline = pipeline.fit([0])
print("Pipeline ejecutado correctamente")
```

```
2025-09-26 01:14:50,220 - DataCleaner - INFO - Fit sen Subcarpet train/
2025-09-26 01:15:00,844 - DataCleaner - INFO - Inicia Verificación de Valores Faltantes en: /content/drive/MyDrive/MIA/Datase
t_origin/train
2025-09-26 01:15:02,803 - DataCleaner - INFO - Inicia Verificación de Outliers en: /content/drive/MyDrive/MIA/Dataset_origin/
train
2025-09-26 01:15:17,110 - DataCleaner - INFO - Fit sen Subcarpet val/
2025-09-26 01:15:17,113 - DataCleaner - INFO - Inicia Verificación de Valores Faltantes en: /content/drive/MyDrive/MIA/Datase
t_origin/val
2025-09-26 01:15:17,996 - DataCleaner - INFO - Inicia Verificación de Outliers en: /content/drive/MyDrive/MIA/Dataset_origin/
val
2025-09-26 01:15:31,965 - DataCleaner - INFO - Fit sen Subcarpet test/
2025-09-26 01:15:31,968 - DataCleaner - INFO - Inicia Verificación de Valores Faltantes en: /content/drive/MyDrive/MIA/Datase
t_origin/test
2025-09-26 01:15:32,643 - DataCleaner - INFO - Inicia Verificación de Outliers en: /content/drive/MyDrive/MIA/Dataset_origin/
test
2025-09-26 01:15:48,910 - DataCleaner - INFO - Fit DataCleaner completado
2025-09-26 01:15:48,911 - DataCleaner - INFO - Transform DataCleaner completado
2025-09-26 01:15:48,912 - FeatureEngineer - INFO - Extracción de características para subcarpet train
2025-09-26 01:15:51,318 - FeatureEngineer - INFO - Extracción de características para subcarpet val
2025-09-26 01:15:51,627 - FeatureEngineer - INFO - Extracción de características para subcarpet test
2025-09-26 01:15:51,968 - FeatureEngineer - INFO - Encoding de categorías
2025-09-26 01:15:52,002 - FeatureEngineer - INFO - Escalamiento de numéricas
2025-09-26 01:15:52,014 - FeatureEngineer - INFO - Fit FeatureEngineer completado
2025-09-26 01:15:52,016 - FeatureEngineer - INFO - Transform FeatureEngineer completado
2025-09-26 01:15:54,097 - YoloDatasetBalancer - INFO - Distribución inicial de clases: {0: 15583, 1: 5101, 2: 354, 11: 820, 1
3: 141, 4: 26, 8: 799, 5: 1032, 7: 54, 10: 8, 6: 42, 9: 15, 12: 23}
2025-09-26 01:15:54,098 - YoloDatasetBalancer - INFO - Fit YoloDatasetBalancer completado
Pipeline ejecutado correctamente
```

Transform:

```
X_transformed = pipeline.transform([0])
```

```
2025-09-26 01:17:14,562 - DataCleaner - INFO - Transform DataCleaner completado
2025-09-26 01:17:14,564 - FeatureEngineer - INFO - Transform FeatureEngineer completado
2025-09-26 01:17:14,572 - YoloDatasetBalancer - INFO - Generando sampler balanceado...
2025-09-26 01:17:14,575 - YoloDatasetBalancer - INFO - Distribución inicial de clases en sampler: {0: 15583, 1: 5101, 2: 354,
11: 820, 13: 141, 4: 26, 8: 799, 5: 1032, 7: 54, 10: 8, 6: 42, 9: 15, 12: 23}
2025-09-26 01:17:14,576 - YoloDatasetBalancer - INFO - oversample=True, undersample=True, factor=2
2025-09-26 01:17:14,582 - YoloDatasetBalancer - INFO - Aplicando oversampling a clases minoritarias...
2025-09-26 01:17:14,611 - YoloDatasetBalancer - INFO - Aplicando undersampling a clase mayoritaria...
2025-09-26 01:17:57,576 - YoloDatasetBalancer - INFO - Reduciendo clase 0 de 139888 a 16 ejemplos
2025-09-26 01:18:45,953 - YoloDatasetBalancer - INFO - Sampler balanceado generado con 62707 ejemplos
2025-09-26 01:18:45,954 - YoloDatasetBalancer - INFO - Sampler listo para entrenamiento
```

```
from collections import Counter
#Para visualizar que tiene el Sampler listo para entrenamiento en YOLO
#Extraemos las clases según los índices del sampler
labels_ = pipeline.named_steps["Balanceo"].labels_

sampler_labels = [labels_[i] for i in X_transformed]

# Contar frecuencia por clase
counts = Counter(sampler_labels)
print(counts)
```

```
Counter({1: 39908, 0: 16156, 13: 3328, 11: 2627, 8: 654, 2: 34})
```